Université Abou Bekr Belkaid

Faculté des Sciences – Tidjani Haddam -

Département d'informatique

TP2 HPC 2025/2026

4 Ingénieur IA

Créer votre propre cluster HPC virtuel

Prérequis

- **Windows 10** pour le système d'exploitation hôte
- **Virtual box** pour la virtualisation
- **Ubuntu Server 20.04** pour le nœud maître et les nœuds de calcul
- **MPICH2** pour la communication de messages
- **NFS** pour le stockage partagé

Fiche TP réalisée par : ABSARI Baghdadi

Module : Calcul Haute Performance (HPC)

Installation du nœud maître

La première étape consiste à installer Ubuntu Server LTS sur une machine virtuelle . Notez que dans cet article, mon nom d'utilisateur est infoia ; vous pouvez bien sûr le modifier.

Voici les paquets nécessaires pour le nœud maître :

```
# Install relevant packages for the master node

$ sudo apt install cmake git build-essential nfs-kernel-server nfs-common
```

Pour faciliter la communication avec les autres nœuds, nous ajoutons leurs noms au fichier hosts. Modifiez le fichier /etc/hosts avec la commande sudo nano /etc/hosts et ajoutez les lignes suivantes au début.

```
127.0.0.1 localhost

# MPI cluster setup

192.168.56.2 masteria

192.168.56.3 node1

192.168.56.4 node2
```

Installer MPICH

MPICH est une implémentation de la norme MPI (Message Passing Interface). C'est la bibliothèque utilisée par les programmes exécutés sur le cluster HPC pour

communiquer entre les nœuds de calcul. Notez qu'OpenMPI est une autre bibliothèque MPI populaire. Voici les commandes pour télécharger, compiler et installer MPICH sur Ubuntu.

```
# Download MPICH
$ wget https://www.mpich.org/static/downloads/4.0/mpich-4.0.tar.gz
$ tar xfz mpich-4.0.tar.gz
$
# Compile MPICH
$ cd mpich-4.0
$ ./configure --disable-fortran
$ make
$ sudo make install
# Check installation
$ mpiexec --version
```

Configuration du serveur NFS

Pour que le cluster puisse exécuter le programme de manière distribuée sur plusieurs nœuds simultanément, il doit être accessible depuis tous les nœuds. Pour cela, le plus simple est de configurer un partage NFS sur le nœud maître, lui-même monté sur les nœuds de l'ordinateur

```
# Create the shared directory on the master node
```

```
$ cd ~

$ mkdir cloud

# Add an entry to the /etc/exports file:

# /home/infoia/cloud *(rw,sync,no_root_squash,no_subtree_check)

$ echo "/home/infoia/cloud *(rw,sync,no_root_squash,no_subtree_check)" | sudo te
e -a /etc/exports

$ sudo exportfs -ra

$ sudo mount -a

$ sudo service nfs-kernel-server restart
```

Installation des nœuds

La première étape consiste à installer Ubuntu Server LTS sur une machine virtuelle. Pour gagner du temps, il est également possible de cloner directement la VM maître. Dans ce cas, pensez à désactiver le nœud maître avant de démarrer la VM worker, car ils auraient la même adresse IP, ce qui générerait des conflits. Voici les paquets nécessaires pour les nœuds :

```
# Install relevant packages for the worker nodes

$ sudo apt install cmake git build-essential nfs-common
```

Configuration réseau

Suivez les mêmes instructions que pour le nœud maître. Modifiez le nom d'hôte et l'adresse IP en fonction du nœud:

- 192.168.56.2 masteria
- 192.168.56.3 node1
- 192.168.56.4 node2

Modifiez les fichiers /etc/hosts avec la commande sudo nano /etc/hosts et ajoutez les mêmes lignes que dans le nœud maître au début du fichier.

```
127.0.0.1 localhost

# MPI cluster setup

192.168.56.2 masteria

192.168.56.3 node1

192.168.56.4 node2
```

Modifier IP

192.168.56.3

Installer MPICH

Pour installer MPICH sur les nœuds worker, suivez les mêmes instructions que pour le nœud maître.

Configuration du client NFS

Create the shared directory on the master node

```
$ cd ~

$ mkdir cloud

# On the workers, mount the shared directory located on the master node

$ sudo mount -t nfs masteria:/home/infoia/cloud ~/cloud

# Check mounted directories

$ df -h

# Make the mounts permanent, add the entry to the /etc/fstab file

$ cat /etc/fstab

# MPI cluster setup

master:/home/infoia/cloud home/infoia/cloud nfs
```

Regrouper tous les nœuds

Pour garantir une communication fluide entre tous les nœuds, nous devons générer et copier les clés SSH entre eux. Sur chaque nœud, exécutez les commandes suivantes :

```
$ ssh-keygen -t ed25519

# Copy to SSH key to all other nodes (except the current node)

$ ssh-copy-id masteria

$ ssh-copy-id node1
```

```
$ ssh-copy-id node2

# For passwordless SSH  maître pour garder connexion

$ eval "$(ssh-agent -s)"

$ ssh-add ~/.ssh/id_ed25519
```

Configuration NTP (serveur + client Ubuntu)

Côté serveur *:*

1. **Installer :**

```
sudo apt install ntp -y
```

2. **Configurer :**

```
sudo nano /etc/ntp.conf
```

- Ajouter tes serveurs de temps :

```
server 0.ubuntu.pool.ntp.org iburst

server 1.ubuntu.pool.ntp.org iburst
```

- Autoriser ton réseau :

```
restrict 192.168.56.0 mask 255.255.255.0 nomodify notrap
```

3. **Redémarrer :**

```
sudo systemctl restart ntp
```

4. **Vérifier :**

```
ntpq -p
```

Côté client :

1. **Installer :**

```
sudo apt install ntp -y
```

2. **Configurer :**

```
sudo nano /etc/ntp.conf

Ajouter ton serveur NTP :

server 192.168.56.2 iburst
```

3. **Redémarrer :**

```
sudo systemctl restart ntp
```

4. **Vérifier la synchro :**

```
ntpq -p
```

Utiliser le cluster HPC

Dans cette section, nous allons écrire un programme simple « Hello World », le compiler et l'exécuter sur le cluster HPC. Créez le fichier main.cpp dans le répertoire partagé et copiez-collez ce programme :

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv)
{    // Initialize the MPI environment
   MPI_Init(NULL, NULL);
   // Get the number of processes
   int world_size;
   MPI_Comm_size(MPI_COMM_WORLD, &world_size);

   // Get the rank of the process
   int world_rank;
   MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

   // Get the name of the processor
```

```
    char processor_name[MPI_MAX_PROCESSOR_NAME];
    int name_len;
    MPI_Get_processor_name(processor_name, &name_len);
    // Print off a hello world message


    printf("Hello world from processor %s, rank %d out of %d processors\n",
           processor_name, world_rank, world_size);


    // Finalize the MPI environment.
    MPI_Finalize();


    return 0;
}
```